

Mitigating Hallucinations in SysML v2 Generation Using LLMs and a Tri-Layered Knowledge Graph Reasoning Framework

Qualis, Richard A.

Dept. of System Engineering and Mathematics.

Florida Institute of Technology

Melbourne FL, USA

ORCID: 0009-0000-8876-4360

Abstract—This paper presents a structured reasoning pipeline that integrates Large Language Models (LLMs) with a tri-layered knowledge graph (KG) framework to automate the generation of SysML v2 modeling artifacts from structured requirements. The approach addresses the persistent challenge of LLM hallucinations in digital and model-based systems engineering (MBSE) by grounding generative outputs in curated knowledge sources. Two core KGs are constructed using a custom Python-based Reasoning Engine: one encodes reusable SysML patterns annotated across nine diagram types, and the other captures domain-specific system models (e.g., aerospace, automotive). A third, system-specific KG is automatically synthesized by parsing capability-linked requirements and aligning them with the foundational KGs and structured prompt templates. These KGs enable context-aware, hierarchical system model generation and inform a dual-parameter prompting strategy: one set is auto injected post-KG construction to preserve domain integrity, and another is derived through our Reasoning Engine over system data prior to LLM invocation. The resulting prompts are validated and refined to produce accurate, high-fidelity SysML v2 representations. This framework offers a scalable path for integrating generative AI into digital engineering (DE) workflows. Ongoing work includes semantic alignment, traceability, prompt optimization, curated dataset with more complex cases, and the addition of a knowledge graph-based memory and planner to enable our reasoner to efficiently execute step-by-step plans—further advancing AI-assisted MBSE.

Index Terms—Large Language Models (LLMs) – Retrieval-Augmented Generation (RAG) – SysML-v2 – Tri-Layer Knowledge Graph – Software Engineering – Hallucination – Model-Based Systems Engineering (MBSE) – Digital Engineering (DE) – LLM Prompt – LLM Reasoner.

I. INTRODUCTION

Modern complex systems such as avionics involve deeply interconnected components, making their development time-consuming and error-prone [10], [11], [20]. As system complexity increases, so does the need for automation. Model-Based Systems Engineering (MBSE), a core pillar of Digital Engineering (DE), has emerged as a solution to fragmented processes [14]. Despite this, MBSE struggles with integration, consistency, and scalability [13], [14].

SysML v2 [1] introduces a textual modeling language designed to improve precision and automation through API-driven workflows [15], [16]. However, its complexity and

usability barriers hinder adoption [18]. Recent progress in large language models (LLMs) such as GPT offers new potential for automating formal modeling. However, these models are prone to hallucinations, outputs that appear valid but are semantically incorrect or contextually unfounded [19], [32]. Hallucination is especially problematic for engineering artifacts, where traceability and correctness are essential.

While hallucinations in natural language and code generation are well-documented, their implications for formal modeling (e.g., SysML v2) remain underexplored. Our study addresses this gap by introducing a structured reasoning pipeline that combines LLMs with a tri-layered knowledge graph (KG) architecture to reduce hallucinations and improve semantic fidelity.

Two research questions drive this work:

RQ-1: How does a tri-layered KG architecture-integrating modeling patterns, system context, and reasoning-affects LLM grounding and reduce hallucinations in SysML v2 generation?

RQ-2: What is the impact of integrating capability-linked requirements into system-specific KGs and using structured, KG-driven prompt templates on the accuracy, traceability, and semantic correctness of LLM-generated SysML v2 models?

We implement a KG-assisted pipeline that supports structured prompt generation and domain-aware reasoning to answer these. Two curated KGs encode reusable SysML patterns [4], [21] and domain models across sectors like aerospace and automotive [3]. A third KG is generated automatically by parsing structured requirements aligned with capability models. These KGs populate structured prompt templates to guide LLM outputs.

This approach significantly reduces hallucinations and increases the quality of generated SysML v2 diagrams. The resulting models demonstrate the potential for scalable, AI-assisted modeling in MBSE workflows. Future directions includes semantic alignment, traceability, prompt optimization, curated dataset with more complex cases, and the addition of a KG-based memory and planner to enable our reasoner to efficiently execute step-by-step plans—further advancing AI-assisted MBSE.

The remainder of this paper is organized as follows: Section

II details the KG architecture and prompting strategy. Section III presents our case study and evaluation. Section IV discusses the results. Section V addresses the research questions. Section VI reviews related work. Section VII concludes the paper.

II. TRI-LAYERED KNOWLEDGE GRAPHS WITH PROMPT TEMPLATE ALIGNMENT

This section introduces our concept of a tri-layered knowledge graph (KG) architecture, aligned with reusable prompt templates to enable automated parameter value injection. This mechanism is a critical component of the structured reasoning pipeline presented throughout the paper, serving as a form of contextual guardrails during LLM inference. We describe the construction of each KG layer using SysML modeling patterns enriched with characterization metadata. An explanation of how this metadata, combined with structured requirements and our Reasoning Engine, supports precise domain alignment. Finally, we detail how the KGs dynamically populate prompt templates, enabling the generation of context-aware, domain-specific SysML v2 artifacts.

A. Reuse Challenges in LLM-Based System Modeling

System engineers often aim to reuse existing models to accelerate early-stage design activities in complex engineering domains [35]. Traditionally, this process begins with capturing requirements manually translated into SysML models using modeling tools such as the SysML v2 Reference Implementation [1]. With the advent of LLMs, engineers now construct structured prompts incorporating system requirements and domain context, and submit these prompts to LLMs for automatic generation of SysML v2 artifacts.

Consider a simple prompt: “Write the SysML v2 text notation for a High-Level System Architecture Diagram for a Satellite System.” While direct, this often results in incomplete subsystem definitions or structurally invalid components that fail to represent the intended system architecture. Engineers may respond by expanding the prompt, e.g., “...and include the Telemetry Subsystem.” However, as the number of subsystems and their interactions increases, so does the complexity of the prompt. These large, manually constructed prompts may exceed the LLM’s context window, resulting in fragmented or inconsistent models, and increasing the likelihood of hallucinations [33]. Moreover, iterative prompt refinements may introduce inconsistencies, particularly in complex model constructs such as package structures. Prompt optimization techniques have proven useful in refining prompts for better LLM performance [22], and as LLMs become integrated into real-world systems engineering workflows, their robustness to prompt perturbations becomes increasingly critical [23].

B. Prompt Templates and Knowledge Graph-Driven Context Enrichment

We propose a unified approach combining reusable prompt templates with a tri-layered KG architecture to mitigate these issues. This pipeline dynamically injects parameter values into prompt templates by leveraging metadata mappings among

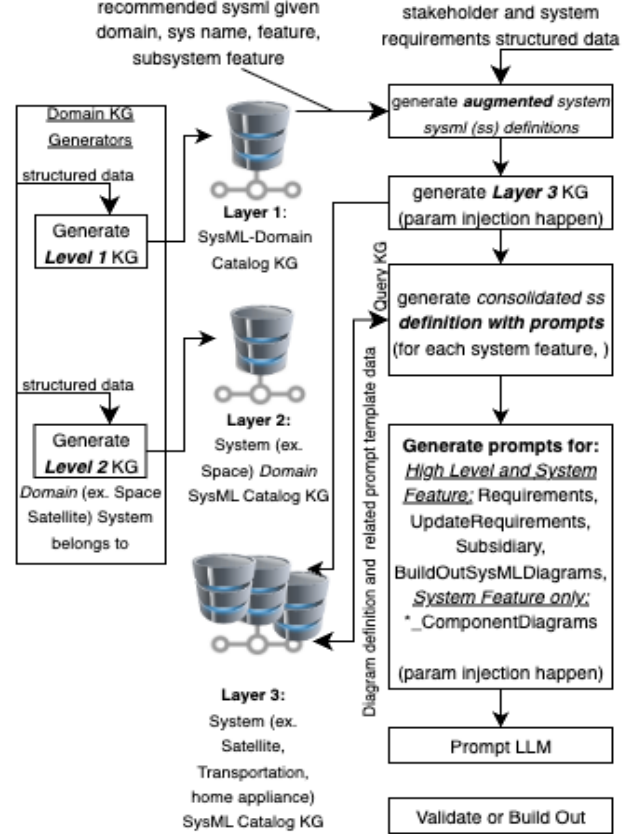


Fig. 1. **Overview of the Tri-Layered Knowledge Graph Framework.** The diagram provides a high-level view of the framework. Boxes on the left represent the KG data loaders for Layers 1 and 2. On the right, natural language requirements are fed into the augmented definition generator, which produces SysML v2 model recommendations for the Layer 1 KG. These recommendations are fused with the requirements to generate data for the Layer 3 KG. The generate-prompts module then creates prompts using templates, augmented definitions, and parameters from Layers 2 and 3. The resulting prompts are used to query the LLM (Prompt LLM), and the generated SysML v2 models can subsequently be validated or further fixed and built out. Layer 3 supports a knowledge graph for each known domain and can be extended to accommodate multi-domain system design scenarios.

system capabilities, diagram types, and domain-specific constructs. The result is a robust context enrichment framework that improves the accuracy, traceability, and domain fidelity of LLM-generated SysML v2 outputs.

1) Layer 1: SysML Characterization Knowledge Graph:

The first layer of the architecture is a SysML-domain-specific KG, Fig. 2, constructed using our data-to-KG loader that processes common SysML v2 modeling patterns. For instance, the High-Level System Architecture Diagram (HLSAD) is typically implemented as a Block Definition Diagram (BDD), which captures the structural hierarchy of major system components [2]–[4].

Each diagram node is tagged with data that described its uses and cases it can or should be used, in an effort to guide prompt construction. For example, HLSAD may be annotated with the “System Overview” tag and linked to a

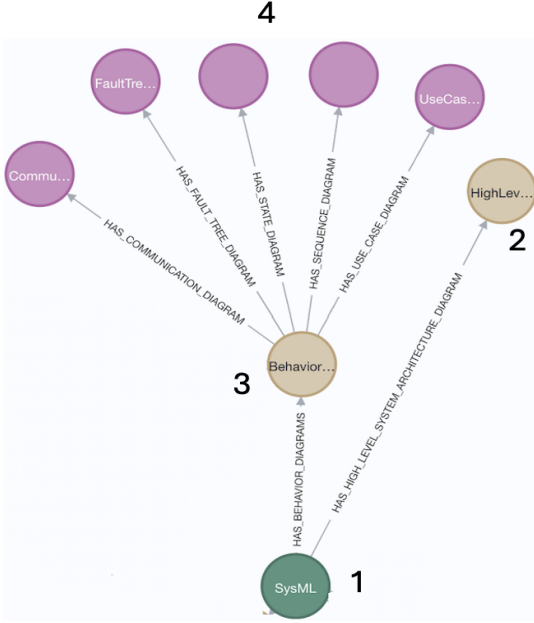


Fig. 2. **Layer 1: SysML Characterization Knowledge Graph:** (1) The root node (green) represents SysML v2. (2) A cluster of nodes that represents high-level diagram types (e.g., HLSAD) that serve as entry points for the system being designed. These nodes are enriched with property values that will be used in Layer 3 prompt templates to ensure subsystem diagrams are correctly linked back to the high-level diagram. (3) A set of nodes corresponding to diagrams recommended for representing 'system features'. (4) A further cluster that represents diagrams associated with the components of each system feature.

BDD representation. Core imports such as `import SI::;` and `import ISQ::;` define structural elements, while optional imports (e.g., `RequirementDerivation::*;`, `Time::DateTime;`) enable richer model expressiveness without violating syntactic integrity. These metadata tags are stored in KG node properties and later used during prompt assembly to ensure syntactic correctness.

2) **Layer 2: Domain-Specific Knowledge Graph:** The second layer (Fig. 3) extends the first by incorporating domain-specific information. This layer fuses generic SysML patterns with context-specific constructs relevant to domains such as aerospace or automotive engineering. For example, a space system's HLSAD may emphasize subsystems like propulsion or telemetry and include behavioral constraints relevant to satellite operations.

By aligning these subsystems with relevant diagram types and modeling expectations, Layer 2 supports semantically enriched prompt generation that reflects the domain's real-world functional and architectural requirements.

3) **Layer 3: System-Specific Knowledge Graph:** The third layer (Figs. ?? and 5) is generated dynamically using structured input data, such as CSV files containing stakeholder and capability-linked requirements. The Reasoning engine parses this data and aligns it with the KG node property values and modeling templates from Layers 1 and 2. The resulting

1	system	diagram
	<code>(:System {system_id: ["SYSTEM_100"],</code>	<code>(:Diagram {record_id: ["RECID_100"],</code>
	<code>name: "Satellite System"})</code>	<code>name: "High Level System Architectur</code>
		<code>e Diagram"))</code>
2	system	diagram
	<code>(:System {system_id: ["SYSTEM_111"],name: "Telemetry"})</code>	<code>(:Diagram {record_id: ["RECID_166"],name: "Communication Diagram"})</code>
	<code>(:System {system_id: ["SYSTEM_111"],name: "Telemetry"})</code>	<code>(:Diagram {record_id: ["RECID_154"],name: "Component Diagram"})</code>
	<code>(:System {system_id: ["SYSTEM_111"],name: "Telemetry"})</code>	<code>(:Diagram {record_id: ["RECID_139"],name: "Sequence Diagram"})</code>
	<code>(:System {system_id: ["SYSTEM_111"],name: "Telemetry"})</code>	<code>(:Diagram {record_id: ["RECID_129"],name: "Data Flow Diagram"})</code>
	<code>(:System {system_id: ["SYSTEM_111"],name: "Telemetry"})</code>	<code>(:Diagram {record_id: ["RECID_107"],name: "High Level System Architecture Diagram"})</code>
	<code>(:System {system_id: ["SYSTEM_111"],name: "Telemetry"})</code>	<code>(:Diagram {record_id: ["RECID_109"],name: "High Level System Architecture Diagram"})</code>

Fig. 3. **Layer 2: Domain-Specific Knowledge Graph in table format** (1) The node represents a Satellite System. For this system, a high-level system architecture diagram (HLSAD) is recommended, along with additional diagrams if it is modeled as a subsystem within a larger system. (2) The Telemetry subsystem is shown (partial view) as part of the Satellite System. If the reasoner framework identifies the Satellite System as the top-level system rather than a subsystem, then an HLSAD would be recommended.

system-specific KG captures the unique characteristics of the target system and serves as the final contextual layer for automated prompt generation. The list below refers to Fig ??:

- **SystemFeature (2):** Represents subsystems (e.g., Orbital Control), each following a parallel structure to the main system node (green) for hierarchical reasoning.
- **HLCapability (3):** Encodes a structured capability description of the system, such as real-time telemetry and signal reliability across payloads.
- **HLDiagrams (4):** Parent of child nodes (4) corresponding to recommended HL diagrams, connected via a `HAS_HIGH_LEVEL_DIAGRAM` relationship.
- **HLRequirementSet (5):** Links to stakeholder (yellow) and system-level (teal) requirement nodes.
- **HLPromptTemplates (6):** Encapsulates reusable prompt templates (linked grey nodes) associated with each diagram type.

This structured naming and hierarchy allows the Reasoning Engine to identify whether a prompt pertains to the system or subsystem level, ensuring accurate prompt construction and diagram generation.

4) **Prompt Template Integration:** Prompt templates are designed to be reusable across systems and domains. Each template includes parameter placeholders such as `[SystemName]` or `[RequirementType]`, which are automatically populated using the tri-layered KG structure; shown in Fig. 6. The Reasoning Engine extracts appropriate values and injects them into the templates, producing well-formed, context-aware prompts for LLM inference.

This dynamic injection mechanism improves prompt quality, reduces manual engineering overhead, and ensures that the generated SysML v2 artifacts are structurally correct,


```

package 'Telemetry Subsystem' {
  private import ScalarValues::*;
  private import ISO::*;
  private import SI::*;

  // =====
  // COMMENT SECTION - UPDATE SUMMARY & IMPLEMENTATION MAP
  // =====
  // Summary of Additions:
  // - Defined internal parts of 'Telemetry Subsystem':
  //   * 'Battery Management Interface' - interfaces with the satellite's battery system
  //   * 'Telemetry Processor' - processes battery health and charging data
  //   * 'Downlink Transmitter' - transmits telemetry data to ground control
  // - Defined capabilities and mapped them to parts via 'performs' relationships:
  //   * Collect Telemetry Data
  //   * Process Telemetry Data
  //   * Transmit Telemetry Data to Ground
  // - Established logical dataflow using connectors
  //
  // Implementation Mapping (suggested model):
  // =====
  // SysML Part                                Implementation Suggestion
  // =====
  // Battery Management Interface              CAN/I2C interface to BMS microcontroller
  // Telemetry Processor                      Onboard flight computer or FPGA unit
  // Downlink Transmitter                     S-band or X-band radio communication
  // =====
  // - Can be further refined in platform architecture (e.g. hardware/software model)
  //
  // Purpose: Enable monitoring of battery health and charge cycles; support real-time
  // system performance status transmission to ground control.

  comment 'diagram' /* Telemetry Subsystem */
  comment purpose about diagram /* Diagram feature description - Detailed view of system
  components and interactions. Provides a detailed breakdown of the system components showing
  how they are organized and interact. */

  comment about 'Telemetry Subsystem' /* Define the Telemetry Subsystem below */

  part def 'Telemetry Subsystem' {
    // === Subsystem Parts ===
    part 'Battery Management Interface' : 'Battery Data Interface' {}
    part 'Telemetry Processor' : 'Data Processing Unit' {}
    part 'Downlink Transmitter' : 'Transmission Unit' {}

    // === Capabilities ===
    capability 'Collect Telemetry Data' {}
    capability 'Process Telemetry Data' {}
    capability 'Transmit Telemetry Data to Ground' {}

    // === Capability Allocation ===
    'Battery Management Interface' performs 'Collect Telemetry Data';
    'Telemetry Processor' performs 'Process Telemetry Data';
    'Downlink Transmitter' performs 'Transmit Telemetry Data to Ground';

    // === Connectors: Logical Data Flow ===
    connector 'BMS_to_Processor' {
      end 'fromBMS' : 'Battery Management Interface';
      end 'toProcessor' : 'Telemetry Processor';
    }

    connector 'Processor_to_Transmitter' {
      end 'fromProcessor' : 'Telemetry Processor';
      end 'toTransmitter' : 'Downlink Transmitter';
    }
  }
}

```

Fig. 7. Auto-generated SysML v2 diagram incrementally extended via our pipeline. The structured prompt directs the LLM to insert the 'Comment Section' in the appropriate location.

key columns, including: `sys_req_id`, `sh_req_id`, `system_feature`, `capability`, `sys_req`, and `sh_req`.

- 2) **Diagram Recommendation via Layer 2 Query:** For each row, the Layer 2 Domain-Specific KG (Fig. 3) is queried to retrieve recommended SysML diagram types relevant to the specified `system_feature`. For example, for the feature 'Telemetry', the recommended diagrams included: Data Flow, Sequence, Component, and Communication diagrams.
- 3) **Information Retrieval from Layer 1:** For each recommended diagram, the Layer 1 SysML Characterization KG is queried to extract information, including required import statements, related diagrams, and semantic annotations. These are used to support both prompt construction and downstream model validation.
- 4) **SysML Definition Generation:** Using the combined data and requirements, the Reasoner generates a SysML system definition for each requirement. These are aggregated per feature into consolidated JSON files (e.g., `Telemetry.json`), enriched with capability, diagram, and requirement metadata.
- 5) **Layer 3 KG Construction:** For each system feature

definition, the Reasoner constructs a KG structure consisting of subject (parent) and object (child) nodes, along with labeled relationships (predicates) to represent semantic links between entities, as shown in Figs. 4 and 5.

- 6) **System Prompt Assembly:** The pipeline retrieves the System node and its associated System Feature nodes from Layer 3 KG. For each, it gathers all associated diagrams and requirement sets. These values are injected into the corresponding prompt templates (Fig. 6), resulting in a unified system-level prompt file. A total of 2,915 prompts were generated for this case study.
- 7) **Diagram Compilation:** The Reasoner merges all requirement diagram fragments into a single SysML v2 model: Satellite System Requirements Diagram.sysml.
- 8) **Validation – Round 1:** The generated file was validated using the official SysML v2 Implementation API [9]. Errors and their corresponding line numbers were logged for further processing.
- 9) **Error Handling – Round 1:** For each error, the Reasoner populated a `report_error_prompt_template`, injecting the erroneous code and error message into the prompt. The LLM was re-invoked to attempt self-repair. Generated outputs were marked with inline comments to support manual inspection (Fig. 7). This correction loop was limited to one pass per error.
- 10) **Model Build-Out:** For each requirement in the validated SysML v2 package, the pipeline loaded the corresponding system model. A `build_out_sysml_diagram` prompt was populated using the system content, requirement text, capability description, and relevant information, and submitted to the LLM. Outputs were captured for subsystem-level build-out (e.g., Fig. 7).
- 11) **Validation – Round 2:** The final diagrams were manually validated for consistency and correctness. Future work should explore the use of the validation API that was released with the SysML v2 Implementation tool set [1].

IV. ASSESSMENT OF THE RESULTS

Although many publicly available examples of SysML v2 models exist, there is currently no verifiable ground truth dataset that pairs natural language stakeholder or system requirements with their corresponding SysML v2 representations. To address this gap, we constructed a manually curated dataset (available at [36]) that can serve as a reference standard. Such an approach is common in early-stage research when modeling languages or tasks and are still emerging (e.g., in the development of early code summarization and natural language-to-code benchmarks). To maximize reliability, we selected small, well-understood systems that allow for precise validation and traceability. The curation process was guided by a domain expert with over a decade of experience in SysML and system modeling.

This evaluation focuses on two primary objectives: (1) verifying the structural validity and cohesion of the generated SysML v2 models, and (2) assessing semantic accuracy and output consistency across multiple LLMs. A central requirement was ensuring diagram navigability and interoperability within SysML v2 visualizers.

Also, while this study emphasized completeness and structural validity over precision, given that missing or malformed elements most directly impair model usability, potentially extraneous elements were still captured through manual expert scoring, and the development of scalable precision metrics remains an identified area for future research.

To assess consistency and repeatability, the framework was tested across multiple LLMs, with each test case executed multiple times under identical structured prompting conditions. Cumulative plots were generated to visualize variation and output stability across runs. Five evaluation categories were studied, this section highlights key findings via Table II and Figures 8 and 9.

A. Evaluation Metrics

Average Structure Score (AStrS): A programmatic estimate of structural correctness in generated diagrams.

$$\text{AStrS} = \frac{1}{N} \sum_{i=1}^N \left(\frac{H(i)}{M(i)} \right) \quad (1)$$

where N is the number of diagrams evaluated, $H(i)$ is the number of structural errors detected in the i -th diagram, and $M(i)$ is the maximum possible structural issues for that diagram. A lower score indicates better structural fidelity.

Average Manual Structure Score (AMSS): A human-evaluated metric reflecting structural correctness.

$$\text{AMSS} = \frac{1}{N} \sum_{i=1}^N \left(\frac{H(i)}{M(i)} \right) \quad (2)$$

where $H(i)$ here refers to the number of human-identified structural issues in the i -th diagram, and $M(i)$ remains the maximum number of expected structural elements. AMSS enables cross-verification with automated scoring (AStrS).

Average Semantic Score (ASemS): Measures inclusion of domain-relevant concepts in each output.

$$\text{ASemS} = \frac{1}{N} \sum_{i=1}^N \left(\frac{C(i)}{E(i)} \right) \quad (3)$$

where $C(i)$ is the count of expected domain-relevant concepts present in the i -th diagram, and $E(i)$ is the total number of expected concepts based on the prompt and requirement. Higher scores indicate stronger semantic fidelity.

Consistency Ratio (CR): Assesses output determinism across repeated LLM generations for the same input.

$$\text{CR} = \frac{1}{N} \sum_{i=1}^N \left(\frac{R(i)}{T(i)} \right) \quad (4)$$

where $R(i)$ is the number of consistent responses and $T(i)$ is the total number of responses for the i -th prompt across multiple runs. CR serves as a proxy for prompt stability and model reproducibility.

TABLE II
SUMMARY OF SYSML v2 GENERATION ACCURACY ACROSS LLMs

Model	AStrS	AMSS	ASemS	CR
claude-3-opus-20240229	3.82	3.53	1.68	2.78
gemini-1.5-flash	4.57	3.5	1.34	2.65
gpt-3.5-turbo	3.44	2.73	2.14	0.0
gpt-4-turbo	4.03	3.54	2.14	0.0
gpt-4o	4.11	3.16	2.21	0.0
mistral-large-latest	4.11	2.97	2.14	2.87

B. Summary and Observations

Strong alignment between AStrS and AMSS was found across most models, validating the reliability of the heuristic scoring method. ASemS scores were consistently lower, suggesting that while LLMs are proficient at generating syntactically correct structures, integrating accurate domain semantics remains more challenging.

The CR metric revealed notable variation in model determinism. Some models, such as GPT-3.5-Turbo, GPT-4o and GPT-4-Turbo, yielded zero repeatability across multiple runs, suggesting high stochasticity or sensitivity to minor prompt fluctuations. By contrast, Gemini-1.5-Flash and Mistral-Large exhibited modest consistency, suggesting greater resilience or a less sensitive decoding strategy.

Prompt type played a significant role in output consistency. High-consistency prompts typically provided tightly constrained or explicitly structured inputs, often leveraging well-populated knowledge graph contexts. Conversely, vague or under specified prompts increased variability, especially for complex subsystem diagrams.

C. Threats to Validity

While our evaluation was carefully designed, several potential threats to validity must be acknowledged. First, the assessment relied on a single domain expert with over a decade of experience in engineering software systems of varying complexity. This experience spans small-scale application systems, complex data processing for satellite space systems, and specialized combat aircraft within the U.S. Department of Defense (DoD), as well as systems in non-DoD industrial contexts. Although this expertise ensures informed and rigorous evaluation, it introduces a potential expert bias, as judgments about model correctness, traceability, and semantic accuracy may reflect the evaluator's domain knowledge and interpretation.

Second, the curated dataset, while manually validated, is limited to small and well-understood systems to facilitate precise validation and traceability. This selection was necessary to

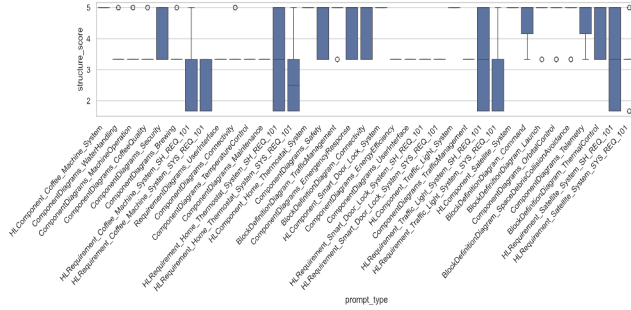


Fig. 8. **Structure Score by Prompt Type:** Box plots showing how closely LLM-generated diagrams conformed to expected structural patterns. Scores range from 1 to 5. Most prompt types achieved high median scores (4–5), with low variance, suggesting consistent structural performance. Wider spreads or lower medians typically correspond to diagram types with deeper nesting or greater syntactic complexity (e.g., BDDs with internal parts). Human-evaluated scores (not shown) closely mirrored the automated heuristic values, confirming scoring reliability.

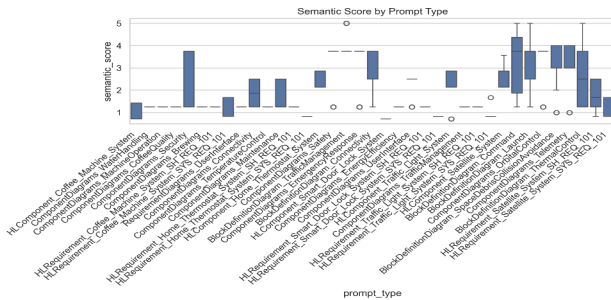


Fig. 9. **Semantic Score by Prompt Type:** This plot measures domain fidelity, how accurately the generated diagrams reflect the intended system capability or feature. Lower overall scores and higher variance suggest semantic embedding remains challenging, especially when domain constraints are weak or inconsistently applied. Higher scores were observed for prompts tightly aligned with the tri-layered KG context.

ensure reliability and repeatability but may limit generalization to larger, more complex, or multi-domain systems.

Third, the framework was tested with a finite set of LLMs and under controlled structured prompting conditions. While this design allows for consistent comparison and assessment of repeatability and mitigation of hallucinations, it may not fully capture performance variability under alternative models, prompts, or real-world operational conditions.

Finally, automated evaluation of SysML v2 model semantic correctness is inherently challenging, as formal metrics for semantic equivalence in complex systems modeling are still emerging. As a result, some aspects of model quality, particularly those related to high-level design intent, rely on human interpretation, which introduces subjectivity.

By explicitly acknowledging these threats, we aim to provide transparency regarding the scope, applicability, and limitations of our findings. Future work will expand on the prompt template to incorporate prescriptive modeling rules, dataset and the evaluation to include larger and multi-domain systems, additional evaluators, and more diverse LLM configurations.

V. DISCUSSION

Overall Summary & Implications for the Framework:

The evaluation results highlight the potential and current limitations of using LLMs to generate SysML v2 artifacts within a knowledge graph-augmented framework. Across test cases, LLMs consistently produced syntactically valid diagrams, particularly when guided by our proposed tri-layered KG architecture. The strong alignment between heuristic and manual structure scores affirms the validity of our automated evaluation methods, enabling scalable assessment.

Semantic fidelity remains a persistent challenge. Models often struggled with abstract or nuanced domain-specific concepts, particularly in cases with sparse or ambiguous prompts. Furthermore, output variability across repeated runs introduced concerns about reproducibility, an important consideration for systems engineering workflows that require traceability or operate in regulated domains.

Addressing RQ-1: In the tri-layered KG architecture, by integrating reusable modeling patterns (Layer 1), domain-specific subsystem and behavior semantics (Layer 2), and system-specific requirement–capability mappings (Layer 3), our pipeline provided LLMs with a multi-tiered semantic framework that constrained outputs, mitigated hallucinations, and ensured structural integrity in the generated SysML v2 models.

Empirical evidence supports this claim. The consistently high structure scores, both automated and human-evaluated, demonstrate that generated artifacts adhered to valid SysML v2 syntax and architectural conventions. Moreover, the minimal deviation between heuristic and manual scores validates the effectiveness of the KG as a structural guide.

Although some hallucinations or off-topic elements persisted, the layered KG narrowed the semantic scope, anchoring the model’s output in well-defined context. This observation suggests that domain-aware constraints can meaningfully reduce error and improve structural coherence when systematically encoded and propagated through the prompt generation process.

Addressing RQ-2: The incorporation of capability-linked requirements into system-specific knowledge graphs, combined with structured, template-driven prompting, had a measurable positive effect on the semantic accuracy and traceability of the resulting SysML v2 models.

While semantic scores remained more variable than structural ones, prompts enriched with requirement–capability context consistently outperformed generic prompts. This indicates that embedding such semantic scaffolding within prompt templates helps LLMs infer and represent domain-relevant elements more accurately.

Moreover, the KG-driven pipeline inherently supports traceability. By linking each generated SysML element back to its originating requirement, feature, or capability node, the framework enables downstream verification, validation, and governance processes. This traceability is especially critical in systems engineering workflows, where model provenance and requirement satisfaction must be rigorously maintained.

These results affirm the value of structured, semantically grounded prompt generation in producing more meaningful, accurate, and auditable model artifacts.

A. Future Work

These findings suggest several avenues for future research. First, automated mechanisms for detecting missing or incomplete SysML v2 elements are urgently needed. In this study, such identification relied on manual inspection, which was both error-prone and difficult to scale across larger models. Programmatic checks—potentially leveraging graph-based consistency rules, semantic validation engines, or ontology-backed verification—would improve efficiency, reproducibility, and the reliability of benchmarking outcomes.

Second, more comprehensive and repeatable benchmarking tools are required. Current evaluations remain limited to structural and semantic alignment, but a more robust framework should also assess coverage (ensuring all requirement types are represented), traceability (linking requirements to design elements), and robustness (evaluating performance under varied prompt formulations and domains). Developing standardized benchmarks and shared datasets would further enable cross-comparison across studies, facilitating progress at the community level.

Third, expanding the curated dataset remains a critical priority. Broader case studies—covering domains such as aerospace, automotive, and cyber-physical systems—along with larger requirement sets and richer semantic embeddings, would strengthen generalizability. Incorporating diverse modeling constructs, such as behavior diagrams and allocation relationships, would also provide a more rigorous testbed for evaluating LLM-driven SysML v2 generation at scale.

Finally, improving determinism and reproducibility remains essential for deployment in safety- and mission-critical contexts. LLM sampling strategies must be more controlled, structured seeding methods should be developed, and repeatable evaluation pipelines must be established. Such advances would reduce variability in model outputs, support certification pathways, and foster trust in the integration of LLM-assisted modeling within Digital Engineering and Model-Based Systems Engineering practices.

VI. RELATED WORK

LLMs have emerged as effective tools for translating natural language into code and formal models, offering significant productivity benefits [26]. However, their hallucination tendency poses risks in high-assurance contexts like systems engineering. For instance, NYU researchers found Copilot-generated code exhibited security flaws in 40% of critical tasks [31].

In MBSE, early SysML model generation automation approaches largely relied on rule-based tools and ontologies. While these methods formalize domain knowledge, they often require extensive manual input, are brittle to system changes, and lack scalability.

Recent efforts aim to integrate more adaptive techniques. For example, [27] proposed an NLP-based pipeline to extract SysML requirement diagrams from natural language, using semantic role labeling and transformation rules. Although promising, this method depends heavily on input quality and rigid linguistic patterns.

Knowledge graphs (KGs) have been explored as a means of enhancing LLM reasoning. A survey in [28] reported that KG integration can reduce hallucinations and improve fidelity, though KG-augmented LLMs remain in early development. Similarly, [37] demonstrated that embedding KG modality within LLMs reduced hallucinations without degrading performance, notably without requiring architectural modifications or retrieval augmentation.

Benchmarking studies underscore the current limitations of LLMs in this space. A SysML v2 benchmark presented in [38] revealed that even advanced models such as GPT-4 performed poorly, highlighting the need for improved methods. Complementary work in [39] proposed corpus-driven approaches for accelerating system design automation, though these were largely non-LLM-centric. Prompt-engineering strategies for SysML v2 multi-source model alignment were investigated in [40], while [41] proposed an LLM-based pipeline for SysML v2 generation, motivated by challenges in corpus scarcity and syntax complexity.

Agent-based methodologies have also been tested. For example, [42] explored LLM-driven SysML v2 generation in industrial contexts using a dataset of only 20 curated items, limiting generalizability and underscoring the knowledge gap in both SysML v2 and MBSE domain semantics. Other notable contributions include metamodel translations between SysML v2 and OWL ontologies using prompt loops [29], and a MagicDraw plugin that generates SysML models via LLMs from natural language inputs [30]. Although the latter focuses on SysML v1, it demonstrates the feasibility of LLM-assisted model synthesis.

Despite this progress, semantic grounding and structural precision remain significant challenges. Our work addresses these gaps by introducing a tri-layered KG architecture that encodes modeling patterns, domain semantics, and system-specific requirement–capability mappings. This design supports scalable, semantically constrained prompting, enabling traceable and reproducible SysML v2 model generation.

VII. CONCLUSION AND OUTLOOK

This paper introduced a tri-layered knowledge graph (KG) framework integrated with structured prompting to enhance Large Language Model (LLM)-assisted generation of SysML v2 artifacts. The proposed approach demonstrates a scalable and extensible method for producing structurally valid and contextually grounded models. Empirical results show that while structural integrity is consistently maintained, validated both heuristically and manually, semantic precision and output determinism remain open challenges, particularly in complex or abstract domains.

Nonetheless, the framework lays a strong foundation for developing robust, traceable, and semantically aligned AI-assisted workflows in Digital Engineering (DE) and Model-Based Systems Engineering (MBSE). Integrating capability-linked requirements, reusable prompt templates, and structured reasoning pipelines significantly improves generated models' accuracy, consistency, and traceability.

REFERENCES

- [1] Object Management Group, "OMG Systems Modeling Language™ (SysML®)," Apr. 2024. [Online]. Available: <https://www.omg.org/spec/SysML/2.0/Beta2/Language/PDF>
- [2] Miro, "The ultimate guide to software architecture diagramming," 2025. [Online]. Available: <https://miro.com/diagramming/what-is-software-architecture-diagramming/>
- [3] S. Friedenthal, A. Moore, and R. Steiner, *A Practical Guide to SysML*, 3rd ed. Boston, MA, USA: MK/OMG Press, 2015, pp. 115–183.
- [4] L. Delligatti, *SysML Distilled: A Brief Guide to the Systems Modeling Language*, 1st ed. Pearson Education, Nov. 2013.
- [5] S. Friedenthal and C. Oster, *Architecting Spacecraft with SysML: A Model-Based Systems Engineering Approach*. CreateSpace, Oct. 2017.
- [6] B. Lackey and L. Liu, "Enriching Knowledge Graphs in Neo4j with Google Enterprise Knowledge Graph," 2023. [Online]. Available: <https://cloud.google.com/blog/products/data-analytics/google-knowledge-graphs-add-context-to-neo4j-graph-database/>
- [7] K. Smith, F. Liu, D. Phanish, H. Chen, R. Chen, and D. Contis, "Research collaboration discovery through Neo4j knowledge graph," in *Proc. Practice and Experience in Advanced Research Computing (PEARC '24)*, New York, NY, USA: ACM, 2024, Art. no. 25, pp. 1–7. [Online]. Available: <https://doi.org/10.1145/3626203.3670539>
- [8] A. R. Miller and D. R. Herber, "Digital engineering transformation of requirements analysis within model-based systems engineering," in *Proc. 10th Annu. World Conf. Soc. Ind. Syst. Eng. (SISE)*, Virtual Conf., Sep. 23–24, 2021. [Online]. Available: <https://ieworldconference.org/content/SISE2021/Papers/MillerAR.pdf>
- [9] California Institute of Technology, "SysML v2 pilot implementation prototyping," 2025. [Online]. Available: <https://github.com/Systems-Modeling/SysML-v2-Pilot-Implementation>
- [10] I. Siddique, "Systems engineering in complex systems: Challenges and strategies for success," *Eur. J. Adv. Eng. Technol.*, vol. 9, no. 9, pp. 61–66, Sep. 2022. [Online]. Available: <https://ssrn.com/abstract=4885918>
- [11] J. Sifakis, "System design automation: Challenges and limitations," *Proc. IEEE*, vol. 103, no. 11, pp. 2093–2103, Nov. 2015. [Online]. Available: <https://doi.org/10.1109/JPROC.2015.2484060>
- [12] F. P. Basso, R. M. Pillat, T. C. Oliveira, F. Roos-Frantz, and R. Z. Frantz, "Automated design of multi-layered web information systems," *J. Syst. Softw.*, vol. 117, pp. 612–637, 2016. [Online]. Available: <https://doi.org/10.1016/j.jss.2016.04.060>
- [13] C. Swickline, T. A. Mazzuchi, and S. Sarkani, "A methodology for developing SoS architectures using SysML model federation," *Syst. Eng.*, vol. 27, no. 2, pp. 368–385, 2024. doi: 10.1002/sys.21727
- [14] A. Ahlbrecht, B. Lukić, W. Zaeske, and U. Durak, "Exploring SysML v2 for model-based engineering of safety-critical avionics systems," in *2024 AIAA DATC/IEEE 43rd Digit. Avion. Syst. Conf. (DASC)*, San Diego, CA, USA, 2024, pp. 1–8. doi: 10.1109/DASC62030.2024.10749311
- [15] C. Zhou, L. Li, Q. Ren, Z. Mu, B. Yu, and B. An, "QVT-based SysML v2 model version difference transformation," in *Proc. 2024 Chin. Autom. Congr. (CAC)*, 2024, pp. 6313–6318. doi: 10.1109/CAC63892.2024.10864958
- [16] J. A. Zhang, B. Bagdatli, and D. N. Mavris, "Leveraging SysML v2 for integration of MBSE and multidisciplinary system development," in *AIAA SCITECH 2023 Forum*, Natl. Harbor, MD, USA, Jan. 2023, Paper AIAA 2023-1895.
- [17] J. K. DeHart, "Leveraging large language models for direct interaction with SysML v2," in *INCOSE Int. Symp.*, vol. 34, pp. 2168–2185, 2024. doi: 10.1002/iis2.13262
- [18] R. Longshore, R. Bell, and R. Madachy, "Leveraging generative AI to build, modify, and query MBSE models," presented at the Acquisition Research Symp., Naval Postgraduate School, Monterey, CA, USA, May 8, 2024. [Online]. Available: <https://dair.nps.edu/bitstream/123456789/52371/1/SYM-AM-24-138.pdf>
- [19] V. Agarwal, Y. Pei, S. Alamir, and X. Liu, "CodeMirage: Hallucinations in code generated by large language models," *arXiv preprint arXiv:2408.08333*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.08333>
- [20] M. Kirshner, "Model-based systems engineering cybersecurity for space systems," *Aerospace*, vol. 10, no. 2, p. 116, 2023. doi: 10.3390/aerospace10020116
- [21] H. P. de Koning, "SysML version 2 approaching industrial use," presented at the ESA MBSE2021 Workshop, Sep. 29–30, 2021. [Online]. Available: <https://indico.esa.int/event/386/contributions/6223/attachments/4266/6445/1015%20%20SysML%20Version%20%20Approaching%20Industrial%20Use.pdf>
- [22] Y. Liu et al., "Beyond prompt content: Enhancing LLM performance via content-format integrated prompt optimization," *arXiv preprint arXiv:2502.04295v1*, 2025. [Online]. Available: <https://arxiv.org/pdf/2502.04295v1>
- [23] G. Chatziveroglou, R. Yun, and M. Kelleher, "Exploring LLM reasoning through controlled prompt variations," *arXiv preprint arXiv:2504.02111v1*, 2025. [Online]. Available: <https://arxiv.org/pdf/2504.02111v1>
- [24] A. Pan, E. Jones, M. Jagadeesan, and J. Steinhardt, "Feedback loops with language models drive in-context reward hacking," *arXiv preprint arXiv:2402.06627v3*, 2024. [Online]. Available: <https://arxiv.org/html/2402.06627v3>
- [25] Anthropic, "Introducing the next generation of Claude," Mar. 4, 2024. [Online]. Available: <https://www.anthropic.com/news/claude-3-family>
- [26] H. Jin, H. Chen, Q. Lu, and L. Zhu, "Towards advancing code generation with large language models: A research roadmap," *arXiv preprint arXiv:2501.11354v1*, Jan. 20, 2025. [Online]. Available: <https://arxiv.org/pdf/2501.11354v1>
- [27] J. Chen, B. Hu, W. Diao, and Y. Huang, "Automatic generation of SysML requirement models based on Chinese natural language requirements," in *Proc. 6th Int. Conf. Electron. Inf. Technol. Comput. Eng. (EITCE)*, New York, NY, USA, 2022, pp. 242–248. [Online]. Available: <https://doi.org/10.1145/3573428.3573470>
- [28] G. Agrawal, T. Kumarage, Z. Alghamdi, and H. Liu, "Can knowledge graphs reduce hallucinations in LLMs? A survey," *arXiv preprint arXiv:2311.07914v2*, Mar. 16, 2024. [Online]. Available: <https://arxiv.org/pdf/2311.07914>
- [29] F. Wawrzik, K. Rafique, T. Urimubenshi, and C. Grimm, "KGG4SE: A knowledge graph generation framework for systems engineering," *Semantic Web Journal*, n.d. [Online]. Available: <https://www.semantic-web-journal.net/system/files/swj3844.pdf>
- [30] S. Saloranta, *From Documents to Models: Adopting MBSE Practices in Complex Embedded Systems and Leveraging Large Language Models*, Master's thesis, Univ. of Turku, Finland, 2024. [Online]. Available: https://www.utupub.fi/bitstream/handle/10024/179433/Saloranta_Samu_opinnayte.pdf
- [31] R. Karri and B. Dolan-Gavitt, "GitHub's Copilot may steer you into dangerous waters about 40% of the time – study," *NYU Tandon School of Engineering*, Aug. 26, 2021. [Online]. Available: <https://engineering.nyu.edu/>
- [32] H. Hu1, Y. Sun, and Q. Zhang, "LRP4RAG: Detecting Hallucinations in Retrieval-Augmented Generation via Layer-wise Relevance Propagation," *arXiv preprint arXiv:2408.15533v2*, Aug. 29, 2024. [Online]. Available: <https://arxiv.org/pdf/2408.15533>
- [33] S. Rasal, "A Multi-LLM Orchestration Engine For Personalized, Context-Rich Assistance," *arXiv preprint arXiv:2410.10039*, Oct. 13, 2024. [Online]. Available: <https://arxiv.org/pdf/2410.10039>
- [34] V. Lopez and L. Thomas, "Metric for Structural Complexity Assessment of Space Systems Modeled Using the System Modeling Language. *Aerospace*", 2022, 9(10), 612. [Online]. Available: <https://doi.org/10.3390/aerospace9100612>
- [35] C. Swickline, T. Mazzuchi, and S. Sarkani, "A methodology for developing SoS architectures using SysML model federation. *Systems Engineering*", 2024, 27(2), 368–385. [Online]. Available: <https://doi.org/10.1002/sys.21727>
- [36] R. Qualis, "RQ MTT Research Dataset," *Hugging Face*, 2025. [Online]. Available: https://huggingface.co/datasets/rqualis/rq_research/tree/main/SysMLv2GenerationTriLayeredKGRReasoner
- [37] V. Chekalina, A. Razzhigaev, E. Goncharova, and A. Kuznetsov, "Addressing Hallucinations in Language Models with Knowledge Graph Embeddings as an Additional Modality", *arXiv preprint*

- arXiv:2411.11531v2, Jan. 14, 2025. . [Online]. Available: <https://arxiv.org/abs/2411.11531>
- [38] D. Jin, Z. Jin, L. Li, Z. Fang, J. Li, X. Chen, and Y. Luo, "A System Model Generalization Benchmark from Natural Language Requirements", arXiv preprint arXiv:2508.03215v1, Aug. 5, 2025. . [Online]. Available: <https://arxiv.org/html/2508.03215v1>
 - [39] M. Hendricks and A. Cicirello, "Text to model via SysML: Automated generation of dynamical system computational models from unstructured natural language text via enhanced System Modeling Language diagrams", arXiv preprint arXiv:2507.06803v2, Jul. 15, 2025. . [Online]. Available: <https://arxiv.org/html/2507.06803v2>
 - [40] Z. Li, S. Husung, and H. Wang, "LLM-Assisted Semantic Alignment and Integration in Collaborative Model-Based Systems Engineering Using SysML v2", arXiv preprint arXiv:2508.16181v1, Aug. 22, 2025. . [Online]. Available: <https://arxiv.org/pdf/2508.16181v1>
 - [41] Y. Bouamra and A. Poisson, "SysTemp: A Multi-Agent System for Template-Based Generation of SysML v2", arXiv preprint arXiv:2506.21608v1, Jun. 20, 2025. . [Online]. Available: <https://arxiv.org/html/2506.21608v1>
 - [42] E. Cibrian, J. Olivert-Iserte, J. Llorens, J. Alvarez-Rodriguez,, "An agent-based approach for the automatic generation of valid SysMLv2 Models in industrial contexts," *Science Direct Computers in Industry Journal*, 2025. 172, [Online]. Available: <https://doi.org/10.1016/j.compind.2025.104350>